Northeastern University
Electrical and Computer Engineering

EECE 2560: Fundamentals of Engineering Algorithms

Fall 2026

Learning Objectives:

- Solve interview-style problems on binary trees using the operations and traversals from the lectures.
- Write clear pseudocode with numbered lines and trace it by hand on a concrete tree.
- Analyse the running time of your pseudocode with the frequency count method and state the final complexity.

Assignment: Binary Tree Interview Questions

Instructions. Answer all 15 questions. Unless a question says otherwise, use the tree built from this array in level order (the node at index i has children at 2i + 1 and 2i + 2):

arr = {5, 12, 8, 21, 3, 17, 9} (n = 7)

For every question, Submit the four parts below.

(a) Pseudocode. Write the algorithm as a procedure with numbered lines, in the same style as the lecture notes (Input, Output, Initial call, then the numbered lines).

(b) Worked example. Trace your pseudocode on the given input. For recursive solutions, list every call with the lines it executes and what it returns. For loops with a queue or stack, give one row per pass showing the queue or stack. State the final output and check it against the expected output.

(c) Frequency count. Give a table with one row per numbered line: Line, Operation, Cost, Frequency (as a formula in n, or h where needed) and Count for the example. Add the rows to get T(n), and check that your formula gives the same total as your example.

(d) Final complexity. Keep the dominant term of T(n) and state the best case, worst case and extra space using Theta notation. Explain in one or two sentences which input gives the worst case.

Counting rules. Every line costs 1. A test such as "if node = NULL" counts every time it is evaluated. A recursive call counts once on the line that makes it. Lines that only end a block (end if, end while, end procedure) are not counted. A tree with n nodes has n + 1 NULL child pointers.

Reference tree. Level 0: 5. Level 1: 12, 8. Level 2: 21, 3, 17, 9. Height h = 2.

Line	Operation	Cost	Frequency	Count for the example
2	(example) test node = NULL	1	2n + 1	15
...				
	Total T(n)			

Grading. Easy questions are worth 5 points, medium 7 points and hard 8 points (100 in total). Each question is marked as: pseudocode 40%, worked example 25%, frequency count 25%, final complexity 10%.

Part A: Easy Questions

Question 1 (5 points) Maximum depth of a binary tree

Return the maximum depth of the tree: the number of edges on the longest path from the root to a leaf (an empty tree has depth -1).

Input: the tree from arr.

Expected output: 2

Twist: Would your frequency count change for a tree that is a single chain of 7 nodes? Explain.

Question 2 (5 points) Sum of all nodes

Return the sum of the values stored in all nodes of the tree.

Input: the tree from arr.

Expected output: 75

Twist: Write it once recursively and once with a queue (level order). Compare the two frequency counts and the extra space each needs.

Question 3 (5 points) Count the nodes that have exactly one child

Return the number of nodes that have a left child or a right child, but not both.

Input: the tree from arr after INSERT(root, 4) (level order rules from the lecture).

Expected output: 1 (node 21, whose only child is 4)

Twist: Explain why a tree built from an array of n elements has exactly one such node when n is even and none when n is odd.

Question 4 (5 points) Are two trees identical?

Given the roots of two trees, return true if they have the same shape and the same value in every position, otherwise false.

Input: tree A from arr; tree B from arr; tree C from {5, 12, 8, 21, 3, 17, 10}.

Expected output: IDENTICAL(A, B) = true; IDENTICAL(A, C) = false

Twist: For IDENTICAL(A, C), at which node does your algorithm first find a difference? What is the best case and the worst case for two trees of n nodes?

Question 5 (5 points) Minimum value in a binary tree

Return the smallest value in the tree. The tree is not a binary search tree, so every node must be checked.

Input: the tree from arr.

Expected output: 3

Twist: How would the algorithm and its complexity change if the tree were a binary search tree?

Part B: Questions

Question 6 (7 points) Invert (mirror) a binary tree

Swap the left and right child of every node, so the tree becomes its mirror image. Return the root.

Input: the tree from arr.

Expected output: level order after inverting: 5, 8, 12, 9, 17, 3, 21

Twist: Give the inorder traversal before (21 12 3 5 17 8 9) and after inverting. What do you notice, and why does it happen?

Question 7 (7 points) Print each level on its own line, and the right side view

Print the nodes of each level on a separate line. Then return the right side view: the last node of each level, which is what you see when looking at the tree from the right.

Input: the tree from arr.

Expected output: levels: [5], [12, 8], [21, 3, 17, 9]; right side view: 5, 8, 9

Twist: Your frequency count will depend on both n and the number of levels h + 1. Show where each term comes from.

Question 8 (7 points) Zigzag (spiral) level order traversal

Visit level 0 from left to right, level 1 from right to left, level 2 from left to right, and so on.

Input: the tree from arr.

Expected output: 5, 8, 12, 21, 3, 17, 9

Twist: Solve it with two stacks, or with a queue plus reversing every second level. Which uses less extra work in your frequency count?

Question 9 (7 points) Is the tree complete?

A tree is complete if every level is full except possibly the last, and the last level is filled from the left. Return true or false. Hint: in level order, once a NULL child has been seen, no real node may appear after it.

Input: tree P from arr; tree Q given in level order with a gap: {5, 12, 8, NULL, 3, 17, 9} (12 has no left child).

Expected output: COMPLETE(P) = true; COMPLETE(Q) = false

Twist: Explain why INSERT and DELETE from the lecture always keep a tree complete.

Question 10 (7 points) Lowest common ancestor

Given two values a and b in the tree, return their lowest common ancestor: the deepest node that has both a and b in its subtree (a node counts as being in its own subtree).

Input: the tree from arr.

Expected output: LCA(21, 3) = 12; LCA(21, 9) = 5; LCA(17, 8) = 8

Twist: Solve it in one of two ways: (1) find both root-to-node paths with FINDPATH and compare them, or (2) a single recursive pass. Compare the frequency counts of the two methods.

Part C: Questions

Question 11 (8 points) Build a tree from its preorder and inorder traversals

Given the preorder and inorder traversals of a tree with distinct values, rebuild the tree and return its root. Hint: the first preorder value is the root; its position in the inorder list splits the left and right subtrees.

Input: preorder = 5, 12, 21, 3, 8, 17, 9; inorder = 21, 12, 3, 5, 17, 8, 9

Expected output: the tree from arr; check: level order 5, 12, 8, 21, 3, 17, 9 and postorder 21, 3, 12, 17, 9, 8, 5

Twist: If you search the inorder list for the root each time, what is the worst-case frequency count? Show how a lookup table (value to inorder index) reduces it to Theta(n).

Question 12 (points) Diameter of a binary tree

The diameter is the number of edges on the longest path between any two nodes. The path does not have to pass through the root.

Input: the tree from arr.

Expected output: 4 (for example the path 21, 12, 5, 8, 17)

Twist: A simple solution calls HEIGHT at every node and takes Theta(n^2) on a chain. Give a solution that computes the height and the diameter in the same pass, and prove with your frequency count that it is Theta(n).

Question 13 (8 points) Serialize and deserialize a binary tree

Write SERIALIZE, which turns a tree into a list of values using preorder and the marker N for every NULL child, and DESERIALIZE, which rebuilds the identical tree from that list.

Input: the tree from arr.

Expected output: 5 12 21 N N 3 N N 8 17 N N 9 N N (DESERIALIZE of this list gives back the tree from arr)

Twist: Why does the list contain exactly 2n + 1 items? Use this fact in the frequency count of both procedures.

Question 14 (8 points) Is the tree a valid binary search tree?

Return true if every node is larger than all values in its left subtree and smaller than all values in its right subtree. Hint: checking only a node against its two children is not enough; pass down an allowed range (low, high).

Input: tree P from arr; tree S given in level order: {9, 5, 17, 3, 8, 12, 21}.

Expected output: ISBST(P) = false; ISBST(S) = true (its inorder is 3, 5, 8, 9, 12, 17, 21)

Twist: Give a second solution that uses an inorder traversal. Compare both frequency counts. What is the best case for each?

Question 15 (8 points) Vertical order traversal

Give the root horizontal distance 0; a left child gets its parent's distance minus 1 and a right child plus 1. Print the nodes column by column from the leftmost distance to the rightmost; inside a column, print top to bottom and left to right (use a level order traversal).

Input: the tree from arr.

Expected output: distance -2: 21 | -1: 12 | 0: 5, 3, 17 | +1: 8 | +2: 9

Twist: Your queue must hold (node, distance) pairs and you need a structure that groups nodes by distance. Include the cost of grouping and printing in the frequency count. How many columns can a tree of n nodes have?